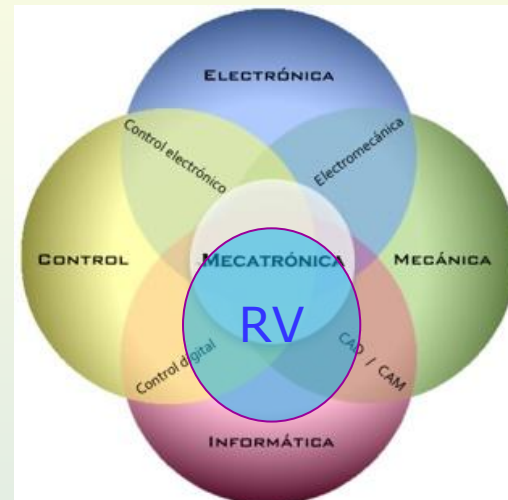


Realidad Virtual

Processing (parte 2)

Ingeniería en Mecatrónica



Contenidos

- Comportamiento dinámico: animación de objetos y personajes, detección de colisión y respuesta, deformaciones geométrica y física.
- Transformaciones geométricas.
- Interacción con dispositivos de E/S. Limitaciones.
- Ejemplos de aplicación en Realidad Virtual y Realidad Aumentada

Animación

- En Processing puede aprovecharse la función `draw()` para realizar animaciones
- Si la aplicación implementa `draw()`, ésta se ejecuta en un `thread` propio
- Tasa de repetición por defecto: 60 Hz
- `frameRate()`: función que define el objetivo de repeticiones por segundo. Su logro depende de las posibilidades del equipamiento específico
- Un elemento importante para lograr este efecto es que cada objeto animado **debe modificar**, en cada frame, alguna **característica sensible** (posición, color, textura, iluminación, etc.)

```

Auto[] misAutos;
Calle miCalle;

void setup() {
    size(400, 250);

    misAutos = new Auto[3];
    miCalle = new Calle(3, color(255, 255, 0));

    for (int i = 0; i < misAutos.length; i++) {
        float x = 0.0;
        float y = 40.0 + i * 80;
        int largo = int(30 + random(40));
        color c = color(random(255), random(255), random(255));
        float veloc_x = 0.1 + random(10);

        misAutos[i] = new Auto(x, y, largo, c, veloc_x);
    }
}

void draw() {
    background(255);

    miCalle.mostrar();

    for (int i = 0; i < misAutos.length; i++) {
        misAutos[i].avanzar();
        misAutos[i].mostrar();
    }
}

class Calle {
    int cantidadCarriles;
    color colorLineas;

    Calle(int cantidadCarriles_, color colorLineas_) {
        cantidadCarriles = cantidadCarriles_;
        colorLineas = colorLineas_;
    }

    void mostrar() {
        stroke(colorLineas);
        strokeWeight(3);

        // Dibuja las divisiones entre carriles
        for (int i = 1; i < cantidadCarriles; i++) {
            float y = i * height / cantidadCarriles;
            line(0, y, width, y);
        }
    }
}

```

```

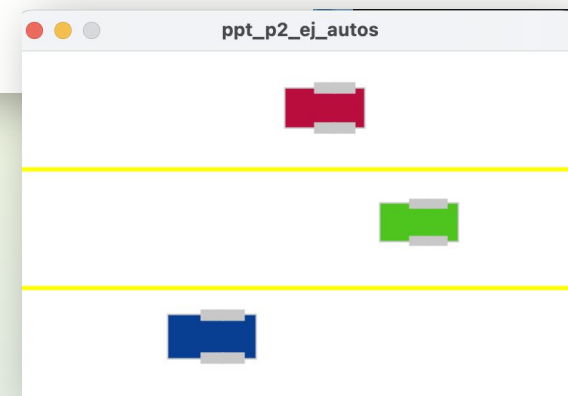
class Auto {
    float x;
    float y;
    int largo;
    color c;
    float veloc_x;

    Auto(float x_, float y_, int largo_, color c_, float veloc_x_) {
        x = x_;
        y = y_;
        largo = largo_;
        c = c_;
        veloc_x = veloc_x_;
    }

    void mostrar() {
        int offset = largo / 4;
        // Cuerpo del auto
        rectMode(CENTER);
        stroke(200);
        strokeWeight(1);
        fill(c);
        rect(x, y, largo, largo / 2);
        // Ruedas relativas al centro
        fill(200);
        rect(x, y - offset, offset, offset / 2);
        rect(x + offset, y - offset, offset, offset / 2);
        rect(x, y + offset, offset, offset / 2);
        rect(x + offset, y + offset, offset, offset / 2);
    }

    void avanzar() {
        x = x + veloc_x;
        if (x > width + largo) {
            x = -this.largo;
        }
    }
}

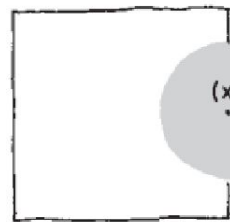
```



Detección de Colisiones

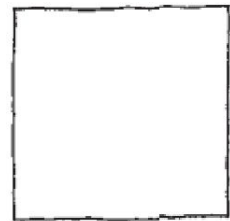
`ellipseMode(RADIUS);`

width



(x, y)

$$x = \text{width}$$



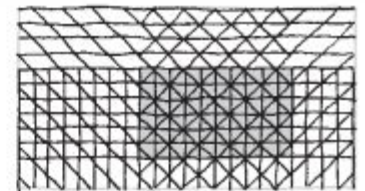
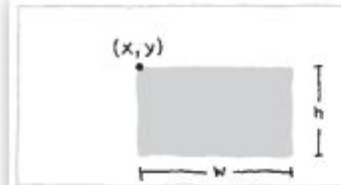
(x, y)

$$x = \text{width} + \text{radius}$$



(x, y)

$$x = -\text{radius}$$

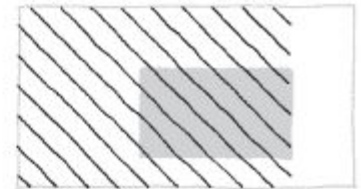


$$(\text{mouseX} > x) \ \&\& \ (\text{mouseX} < x + w) \ \&\& \\ (\text{mouseY} > y) \ \&\& \ (\text{mouseY} < y + h)$$

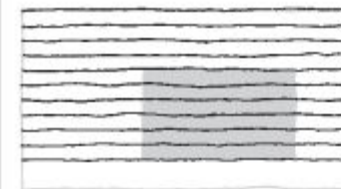
Expresión obtenida a partir de la combinación de las condiciones de frontera



$$\text{mouseX} > x$$



$$\text{mouseX} < x + w$$



$$\text{mouseY} < y + h$$



$$\text{mouseY} > y$$

Ejemplo con Colisiones



```
// Considerando el efecto de g
// Posicion
float x, y;
// Velocidad
float vx, vy;
// Diametro de la bola
int diametro = 20;
// Gravedad
float gravedad = 0.5;

void setup() {
  size (600, 300);
  fill(255);
  noStroke ();
  // Bola en la mitad superior
  x = width/4;
  y = diametro/2;
  // Velocidad inicial
  vx = vy = 1.0;
  // Relleno inicial
  background (0);
}
```

```
void draw() {
  fill(0, 20) ;
  rect(0, 0, width, height);
  fill (255) ;
  // Detecta colision y rebota
  if (x + diametro/2 > width - 1) {
    vx *=-1; x = width - 1 - diametro/2;
  }
  else
    if (x - diametro/2 < 0) {
      vx *=-1; x = diametro/2;
    }
  if (y + diametro/2 > height - 1) {
    vy *=-1; y = height - 1 - diametro/2;
  }
  else
    if (y - diametro/2 < 0) {
      vy *=-1; y = diametro/2;
    }
  // Actualizar posicion
  vy += gravedad;
  x += vx;
  y += vy;
  // Dibujamos
  ellipse (x, y, diametro, diametro);
}
```

Consideraciones de «Realidad»

- Por ejemplo, considerando pérdidas de energía por rozamiento con el aire

```
// Actualizar posicion amortiguada
vy += gravedad;
vy *= 0.99;
vx *= 0.997;
if (abs(vy)<0.1)
    vy=0;
if (abs(vx)<0.1)
    vx=0;

x += vx;
y += vy;

// Dibujamos
ellipse (x, y, diametro, diametro);
```



- Hay que buscar las ecuaciones más apropiadas en cada caso (igualdad o semejanza a la realidad)

Interacción

- Processing admite 2 mecanismos para controlar la interacción con el usuario:
 - Mediante la consulta de **variables del sistema**
 - Mediante la definición de **funciones de respuesta a eventos**
- El uso de un mecanismo u otro depende de las necesidades de la aplicación. Pueden combinarse
- Mecanismo de consulta de variables del sistema:
 - Se realiza generalmente en la función draw()
 - No garantiza la detección de todos los eventos
- Mecanismo por eventos:
 - Usa la cola de eventos y de funciones de respuesta (tratamiento) a los mismos
 - Es apropiado para todas las detecciones

Interacción con el Mouse

- Variables [de Estado] del sistema

- `mouseX`, `mouseY`

- Almacenan siempre la posición actual del mouse (coordenadas x e y respectivas)

- `pmouseX`, `pmouseY`

- Almacenan la posición (coord. X e y) del mouse en el frame anterior

- `mousePressed`

- toma valor True o False cuando un botón del ratón se encuentra o no pulsado

- `mouseButton`

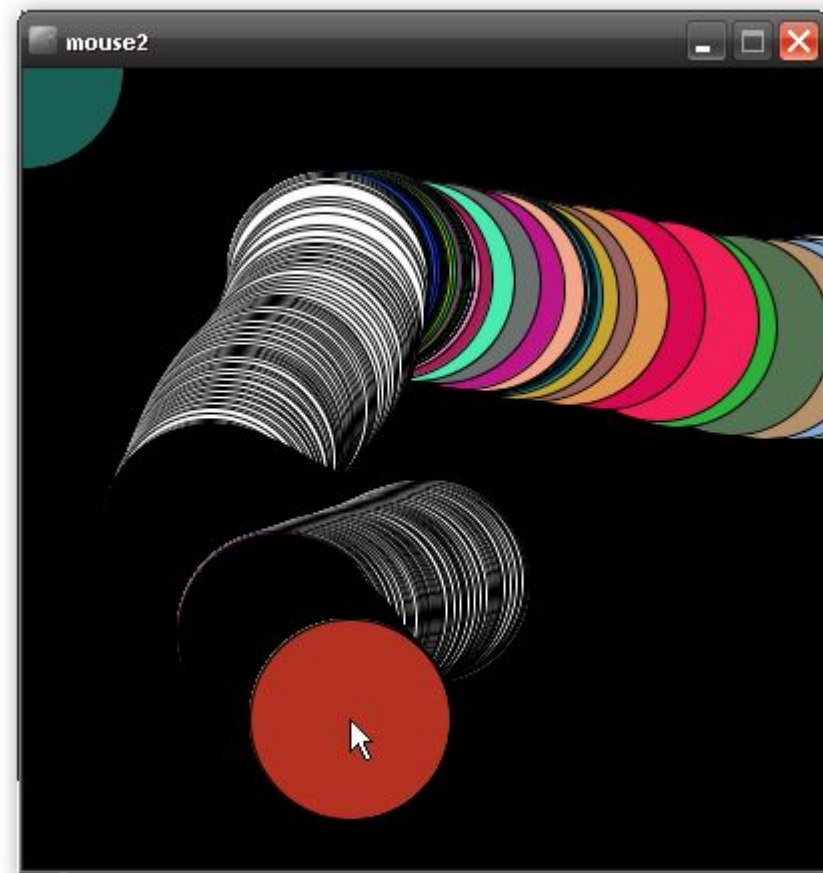
- toma valor LEFT, RIGHT o CENTER en función del botón pulsado. Funciona combinado con `mousePressed`

Ejemplo: interacción mediante mouse

```
void setup(){
  size(400,400);
  background(0);
  smooth();
}
void draw(){
  fill(random(255),random(255),random(255));

  if(mousePressed){
    if(mouseButton== LEFT)
      fill(255,255,255);
    else if(mouseButton== RIGHT)
      fill(0,0,0);
  }

  ellipse(mouseX, mouseY, 100,100);
}
```



Interacción con Teclado

- Variables [de Estado] del sistema
 - `keyPressed`:
 - toma valor TRUE si una tecla ha sido pulsada
 - `key`
 - Toma el valor (caracter) de la tecla básica pulsada
 - Reconoce las teclas BACKSPACE, TAB, ENTER, RETURN, ESC y DELETE
 - Puede tomar el valor especial CODED
 - `keyCode`:
 - indica la pulsación de alguna tecla especial: UP, DOWN, LEFT, RIGHT, ALT, CONTROL, SHIFT.
 - Su uso debe combinarse con `keyPressed` y el valor especial CODED

Funciones de [respuesta a] Eventos

- La definición de cada función se realiza añadiendo programa una nueva función
- Son ejecutadas siempre que se produce el evento asociado
- Tienen nombres específicos, sin parámetros y retornan void
- Es habitual que en el cuerpo de la función se consulte a las variables del sistema

Funciones de Mouse y Teclado

- `void mousePressed()`: invocada al pulsar un botón
- `void mouseReleased()`: invocada al liberar un botón (luego del 1er click)
- `void mouseClicked()`: luego de realizar una pulsación completa, es decir, presión y liberación
- `void mouseMoved()`: llamada después de un movimiento de la posición del ratón si que ningún botón se encuentre pulsado
- `void mouseDragged()`: llamada después de un movimiento de la posición del ratón con algún botón pulsado
- `void keyPressed()`: invocada luego de la pulsación de una tecla
- `void keyReleased()`: invocada al soltar una tecla (luego de haber sido pulsada)
- `void keyTyped()`: llamada después de pulsar y liberar la tecla

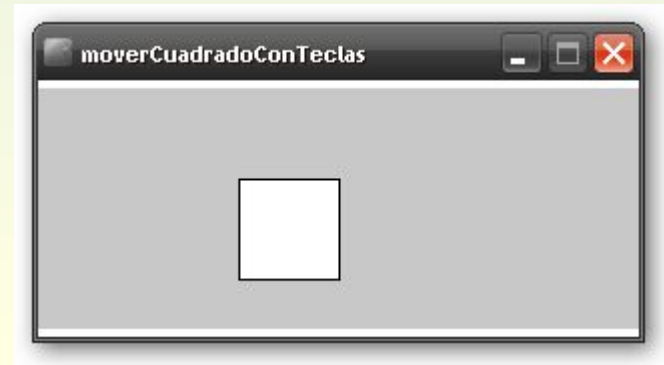
Ejemplo: eventos de teclado

```
int x = 100;

void setup() {
  size(300, 120);
}

void draw() {
  background(200);
  rect(x, 45, 50, 50);
}

void keyPressed() {
  if (key == CODED) {
    if (keyCode == LEFT)
      x--;
    else if (keyCode == RIGHT)
      x++;
  }
  else
    if ((key == 'A') || (key == 'a'))
      fill(0, 0, 255);
    else
      if ((key == 'V') || (key == 'v'))
        fill(0, 255, 0);
}
```

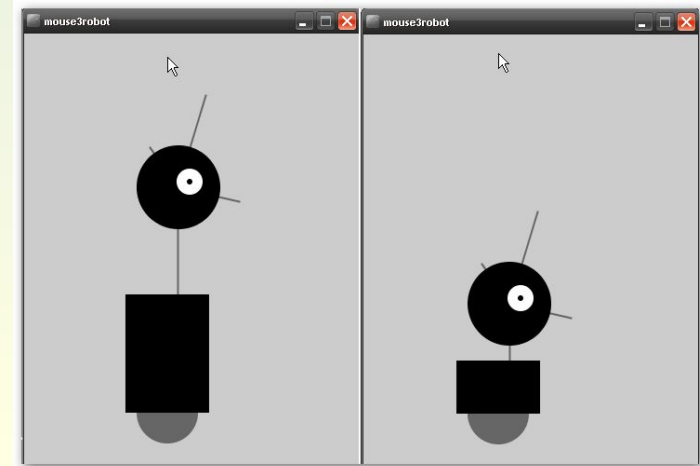


Ejemplo: eventos del mouse

```
float x = 60;           // coordenada
float y = 440;          // coordenada
int radioCabeza = 45;   // Radio de cabeza
int altoCuerpo = 160;   // alto del cuerpo
int altoCuello = 70;    // alto el cuello
float freno = 0.02;

void setup() {
  size(360, 480);
  smooth();
  strokeWeight(2);
  ellipseMode(RADIUS);
}

void draw() {
  float ny = y - altoCuerpo - altoCuello - radioCabeza;
  background(204);
  // cuello
  stroke(102);
  line(x+12, y-altoCuerpo, x+12, ny);
  // antena
  line(x+12, ny, x-18, ny-43);
  line(x+12, ny, x+42, ny-99);
  line(x+12, ny, x+78, ny+15);
  // cuerpo
  noStroke();
```



```
void mousePressed(){
  altoCuello = 16;
  altoCuerpo = 90;
}

void mouseReleased(){
  altoCuello = 70;
  altoCuerpo = 160;
}

void mouseMoved(){
  x += (mouseX - x) * freno;
}
```

Modos 3D en Processing

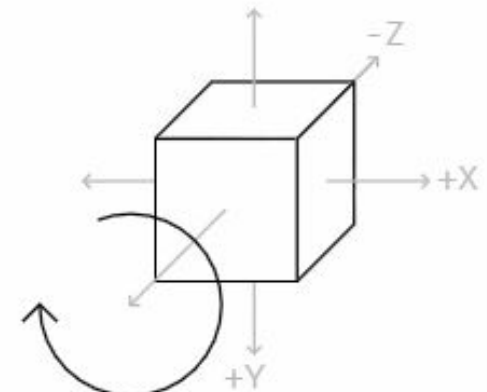
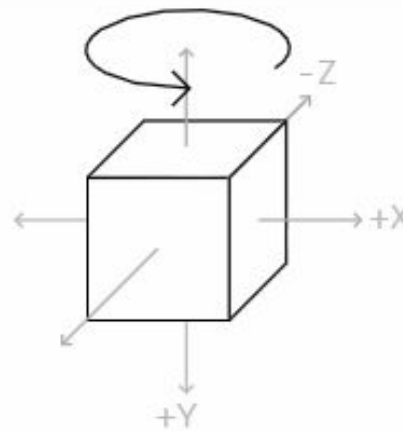
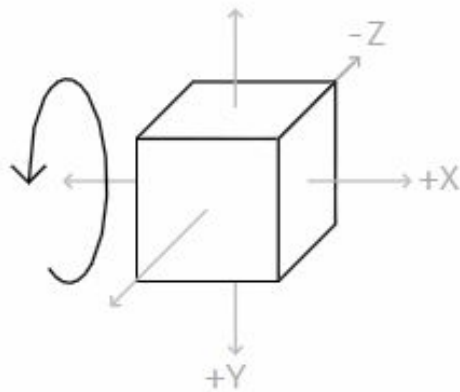
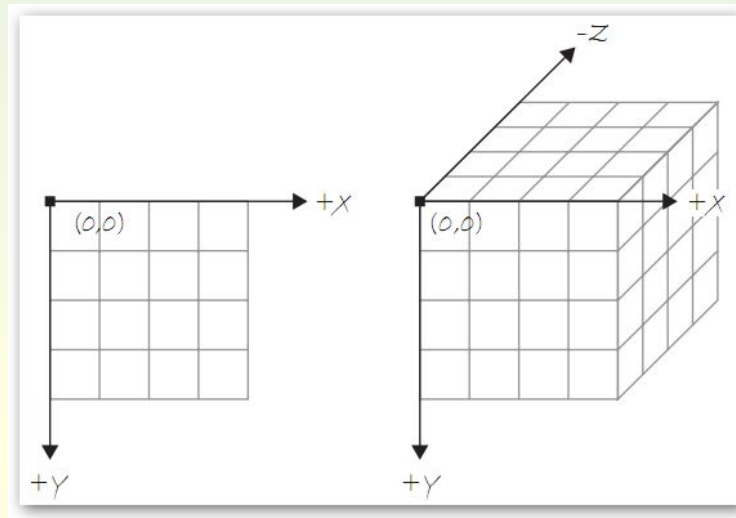
- El **modo** le indica a Processing qué hacer con las escenas cuando renderiza éstas a la ventana.
 - Indicado en la función **size()** como tercer argumento
 - **size (200, 200, P3D)** o **size (200, 200, OPENGGL);**
 - Si OPENGGL, en el sketch debe hacerse **import processing.opengl.*;**
- **P3D:**
 - Es un renderer 3D desarrollado para Processing.
 - Posee la función anti-aliasing activado por defecto.
 - Aplicable en casos de baja/mediana complejidad o de baja/mediana resolución de pantalla
- **OPENGGL:**
 - Es un renderer 3D que emplea aceleración por hardware.
 - Requiere una placa gráfica compatible con OpenGL.
 - Recomendable casos de mediana/alta complejidad o de mediana/alta resolución de pantalla

Primitivas 3D y transformaciones afines

- `point(x, y, z)` `line(x1, y1, z1, x2, y2, z2)`
- `box(tamaño)` o `box(ancho, alto, profundidad)`
- `sphere(radius)`
- `beginShape()` `endShape()`
 - Con `Vertex(V)` `vertex(x, y, z)` `vertex(x, y, z, u, v)`
- Otras disponibles también en 2D

- `translate(tx, ty, tz)`
- `scale(sx, sy, sz)`
- `rotate(ángulo, x, y, z)`
- `rotateX(ángulo)` - `rotateY(ángulo)` - `rotateZ(ángulo)`
- `pushMatrix()` - `popMatrix()`
- `resetMatrix()` `printMatrix()`
- `applyMatrix()`

3D con Processing



Figuras a partir de Vértices

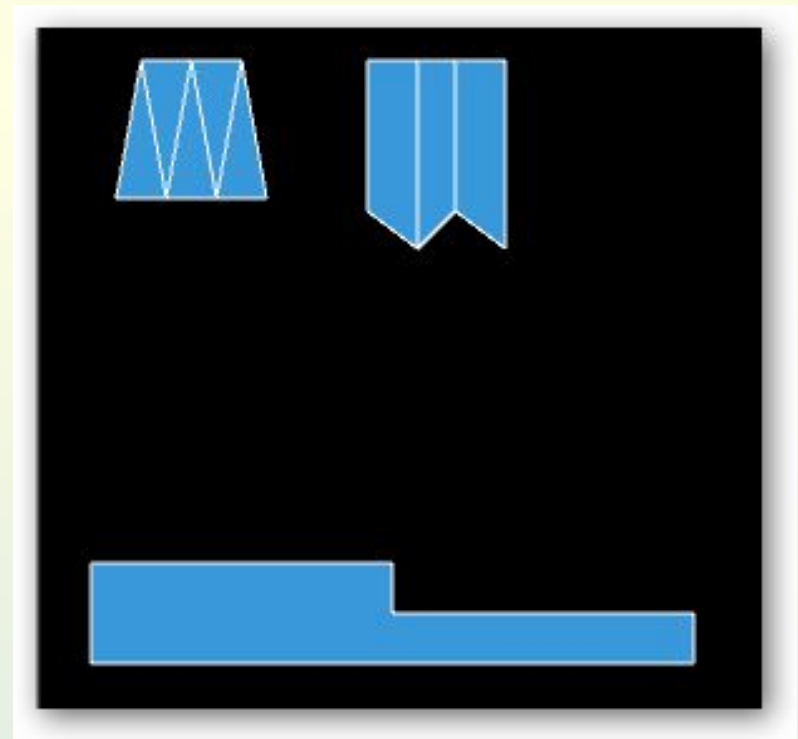
- `vertex(V)` `vertex(x, y, z)` `vertex(x, y, z, u, v)`
- Parámetros
 - `V` `float[]`: array de vértices
 - `x, y, z` `float`: coordenadas de 1 vértice de la figura
 - `u, v` `float`: coordenadas horizontal y vertical mapa de textura

```
size(400, 400, P3D);  
smooth();  
stroke(255,255,255);  
background(0);  
fill(57, 152, 218);
```

```
beginShape(TRIANGLE_STRIP);  
vertex(30, 75); vertex(40, 20); vertex(50, 75);  
vertex(60, 20); vertex(70, 75); vertex(80, 20);  
vertex(90, 75);  
endShape();
```

```
beginShape(QUAD_STRIP);  
vertex(130, 20); vertex(130, 80); vertex(150, 20);  
vertex(150, 95); vertex(165, 20); vertex(165, 80);  
vertex(185, 20); vertex(185, 95);  
endShape();
```

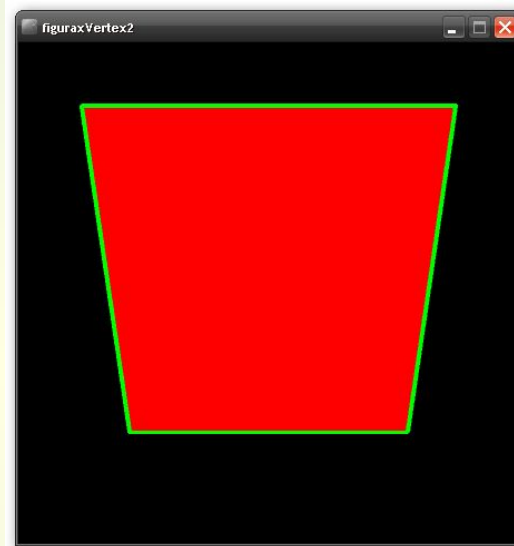
```
beginShape();  
vertex(20, 220); vertex(140, 220); vertex(140, 240);  
vertex(260, 240); vertex(260, 260); vertex(20, 260);  
endShape(CLOSE);
```



PShape en 3D

■ PShape

- isVisible()
- setVisible()
- disableStyle()
- enableStyle()
- beginContorn()
- endContorn()
- beginShape()
- endShape()
- getChildCount()
- getChild()
- addChild()
- getVertexCount()
- getVertex()
- setVertex()
- translate()
- rotateX()
- rotateY()
- rotateZ()
- rotated()



```
PShape s;  
  
void setup() {  
  size(450, 450, P3D);  
  s=(new Cuadro()).getCuadro();  
  background(0);  
  shape(s, 100, 100);  
}  
  
class Cuadro{  
  PShape s;  
  
  Cuadro(){  
    s = createShape();  
    s.beginShape();  
    s.vertex(0, 0, 100);  
    s.vertex(250, 0, 100);  
    s.vertex(250, 250, 0);  
    s.vertex(0, 250, 0);  
    s.endShape(CLOSE);  
    s.setStroke(color(0, 255, 0));  
    s.setStrokeWeight(4);  
    s.setFill(color(255, 0, 0));  
  }  
  
  PShape getCuadro(){  
    return s;  
  }  
}
```

Iluminación

```
void draw() {  
  background(0);  
  translate(100, 100, 0);  
  if (keyPressed) {  
    if ((key == 'L') || (key == 'l'))  
      lights();  
    else  
      if ((key == 'A') || (key == 'a'))  
        ambientLight(255, 200, 255);  
      else  
        if ((key == 'D') || (key == 'd'))  
          directionalLight(0, 255, 0, 0, -1, 0);  
        else  
          if ((key == 'S') || (key == 's')) {  
            // de color rojo: 255, 0, 0  
            // ubicada en el frente de la ventana Processing  
            // width, height, 400  
            // apuntando hacia atras: 0, 0, -1  
            // con un pequeño ángulo de PI/4  
            // Con intensidad (concentration): 2  
            spotLight(255, 0, 0, width, height, 400, 0, 0, -1, PI/4, 2);  
          }  
        else  
          if ((key == 'P') || (key == 'p'))  
            pointLight(255, 0, 0, width/2, height/2, 400);  
      }  
    noStroke();  
    fill(255);  
    sphere(50);  
  }  
}
```



Texturas

```
PImage img;

void setup() {
  size(550, 550, P2D);
  img = loadImage("perseguido.png");
  textureMode(NORMAL);
}

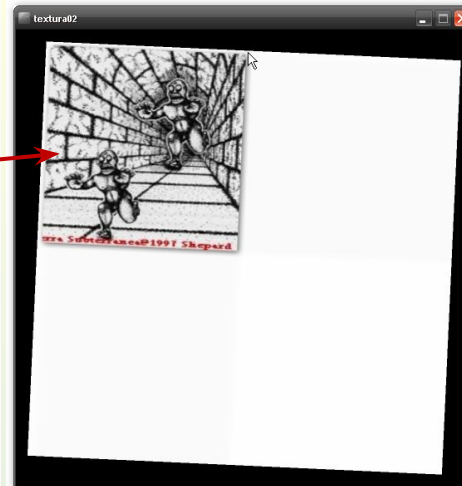
void draw() {
  background(0);
  translate(width/2, height/2);
  rotate(map(mouseX, 0, width, -PI, PI));
  if (mousePressed) {
    textureWrap(REPEAT);
  }
  else {
    textureWrap(CLAMP);
  }
  beginShape();
  texture(img);
  vertex(-250, -250, 0, 0);
  vertex(250, -250, 2, 0);
  vertex(250, 250, 2, 2);
  vertex(-250, 250, 0, 2);
  endShape();
}
```

- PImage / loadImage(imagen)

- textureMode(IMAGEN)

- textureMode(NORMAL)

- textureWrap()



- texture(imagen)

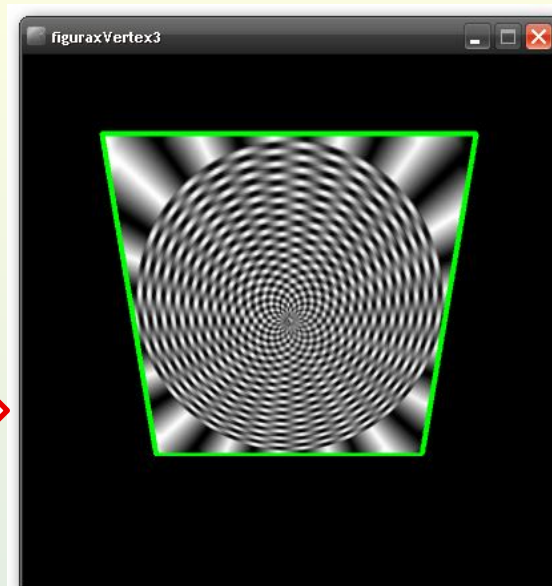
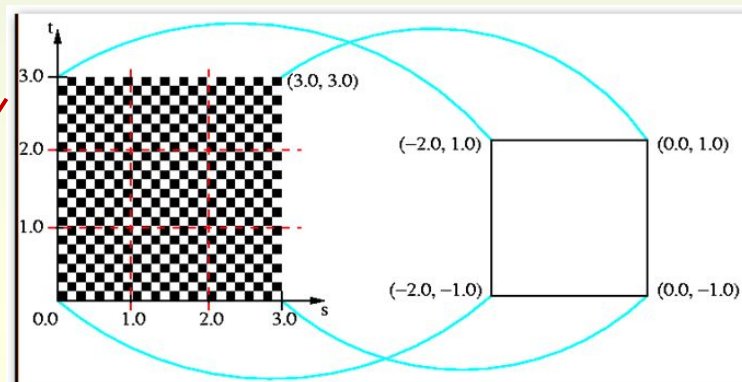
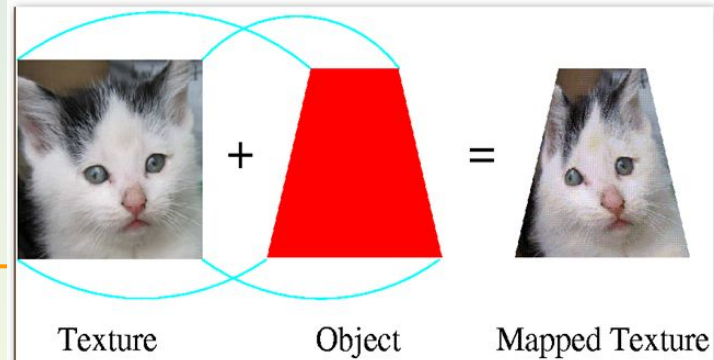


Mapeo de textura

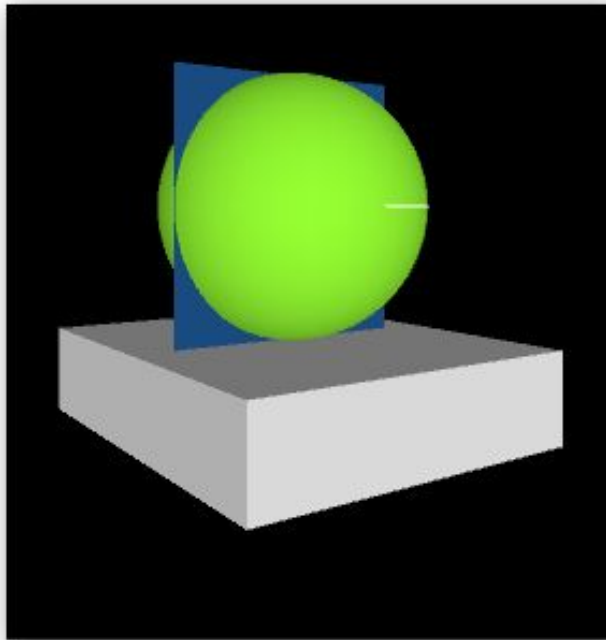
```
void setup() {  
  size(400, 400, P3D);  
  dibujo = loadImage("ilusion.jpg");  
  textureMode(IMAGE); // alternativa NORMAL  
}
```

```
void draw() {  
  background(0);  
  translate(width / 2, height / 2);  
  stroke(color(0, 255, 0));  
  strokeWeight(4);  
  crearCuadro();  
  stroke(255, 0, 0);  
  strokeWeight(2);  
  line(0, 0, -150, 0, 0, 150);  
}
```

```
void crearCuadro() {  
  beginShape();  
  texture(dibujo);  
  vertex(-100, -100, 100, 0, 0);  
  vertex( 100, -100, 100, dibujo.width, 0);  
  vertex( 100, 100, 0, dibujo.width, dibujo.height);  
  vertex(-100, 100, 0, 0, dibujo.height);  
  endShape(CLOSE);  
}
```



Dibujar y Transformar 3D



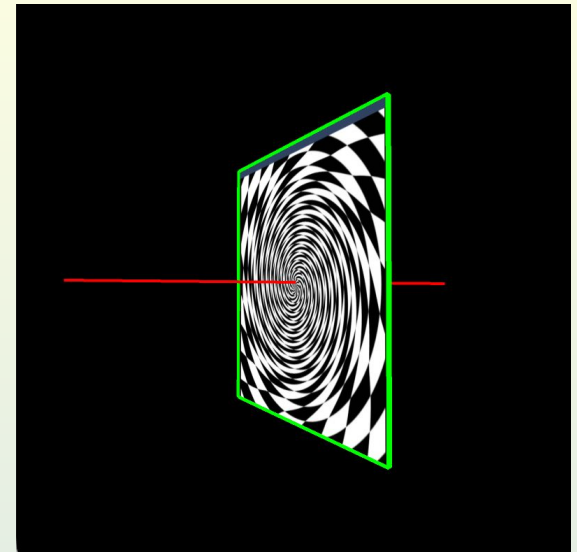
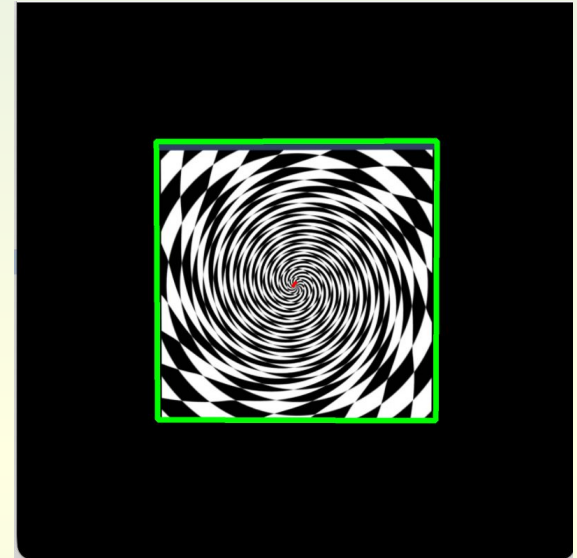
```
int x,y;
void setup() {
  size(400, 400, P3D);
  x=mouseX; y=mouseY;
}

void draw() {
  background(0);
  lights();
  noStroke();
  fill(233);
  pushMatrix();
  translate(x, y, -500);
  rotateY(PI / 6); // Rotar alrededor de eje Y
  box(400, 100, 400); // Dibuja la caja
  pushMatrix();
  popMatrix();
  translate(0, -200, 0); // Ubica la esfera
  fill(150,255,50);
  sphere(150); // Dibuja la esfera sobre la caja
  fill(50,150,255);
  rect(-150, -150, 300, 300); // plano de corte
  stroke(255);
  line(0, 0, -200, 0, 0, 200);
  popMatrix();
}

void mouseMoved(){
  x=mouseX;
  y=mouseY;
}
```

Movimiento en 3D con texturas

```
PImage dibujo;  
PShape cuadro;  
  
float rotx = 0; // Probar con PI/4  
float roty = 0; // Probar con PI/4  
  
void setup() {  
  size(400, 400, P3D);  
  
  dibujo = loadImage("ilusion.jpg");  
}  
  
void draw() {  
  background(0);  
  
  // La posición del mouse controla la inclinación del plano  
  rotx = map(mouseY, 0, height, PI / 3, -PI / 3);  
  roty = map(mouseX, 0, width, -PI / 3, PI / 3);  
  
  translate(width / 2, height / 2);  
  rotateX(rotx);  
  rotateY(roty);  
  
  stroke(0, 255, 0);  
  strokeWeight(4);  
  crearCuadro();  
  
  stroke(255, 0, 0);  
  strokeWeight(2);  
  line(0, 0, -150, 0, 0, 150);  
}  
  
void crearCuadro() {  
  beginShape();  
  texture(dibujo);  
  
  vertex(-100, -100, 0, 0);  
  vertex(100, -100, 0, dibujo.width, 0);  
  vertex(100, 100, 0, dibujo.width, dibujo.height);  
  vertex(-100, 100, 0, 0, dibujo.height);  
  
  endShape(CLOSE);  
}
```



Construir 3D

```
class Ladrillo {
    Punto3D[] vertices = new Punto3D[24];
    float w, h, d;

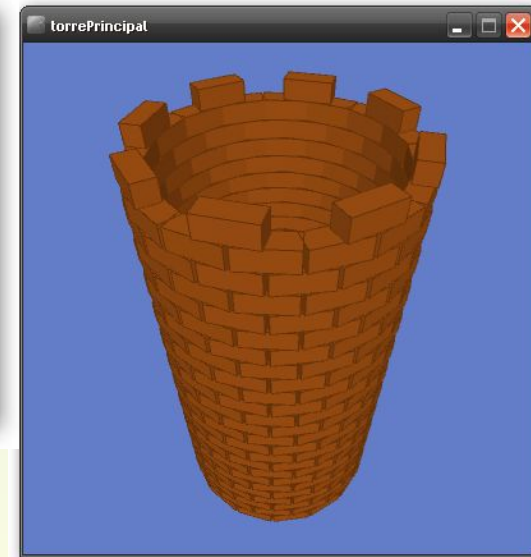
    Ladrillo() {
    }

    Ladrillo(float w, float h, float d) {
        this.w = w;
        this.h = h;
        this.d = d;
        // Ladrillo compuesto de 6 cuadrículas
        //frente
        vertices[0] = new Punto3D(-w/2, -h/2, d/2);
        vertices[1] = new Punto3D(w/2, -h/2, d/2);
        vertices[2] = new Punto3D(w/2, h/2, d/2);
        vertices[3] = new Punto3D(-w/2, h/2, d/2);
        //izquierda
    void create() {
        // dibuja el ladrillo
        for (int i=0; i<6; i++) {
            beginShape(QUADS);
            for (int j=0; j<4; j++) {
                vertex(vertices[j+4*i].x,
                    vertices[j+4*i].y, vertices[j+4*i].z);
            }
            endShape();
        }
    }
}
```

```
class Punto3D {
    float x, y, z;

    Punto3D() {
    }

    Punto3D(float x, float y, float z) {
        this.x = x;
        this.y = y;
        this.z = z;
    }
}
```

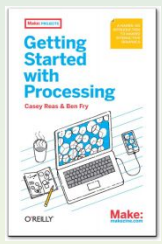
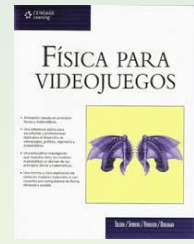
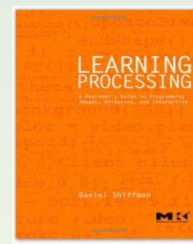
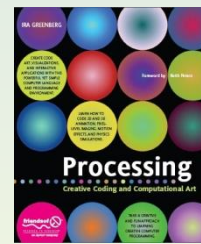
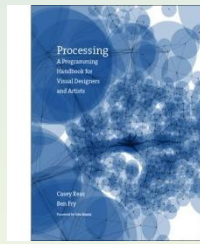


```
float ladrillosPorHilera = 16.0; // debe ser par
float hilerasDeLadrillos = 25.0;
Ladrillo[] ladrillos = new Ladrillo[int(ladrillosPorHilera * hilerasDeLadrillos)];
float largoLadrillo = 60, altoLadrillo = 25, anchoLadrillo = 25;
float radio = 150.0;
float angulo = 0;

void setup() {
    size(400, 400, P3D);
    for (int i=0; i<ladrillos.length; i++) {
        ladrillos[i] = new Ladrillo(largoLadrillo, altoLadrillo, anchoLadrillo);
    }
}

void draw() {
    background(100, 125, 200);
    float tempX = 0, tempY = 0, tempZ = 0;
    fill(175, 87, 20);
    stroke(100, 50, 10);
    // iluminacion básica
    lights();
    translate(width/2, height*1.2, -550);
    // se inclina la torre para ver el interior
    rotateX(radians(-45));
    // suave rotación de la torre
    rotateY(frameCount*PI/60);
    for (int i=0; i<hilerasDeLadrillos; i++) {
        // incrementa la hilera
        tempY-=altoLadrillo;
        // agrega uniones entre ladrillos
    }
}
```

Bibliografía



■ Libros

- REAS, Casey y FRY, Ben (2007): "Processing. A programming handbook for visual designers and artists". The MIT Press Cambridge, England.
- GREENBERG, Ira (2007): "Processing. Creative Coding and Computational Art", Ed. Friends of, Springer, N.Y. [[capítulos 9 a 13](#)]
- SHIFFMAN, Daniel (2008): "Learning Processing: A Beginner's Guide to Programming Images, Animation, and Interaction". Ed. Morgan Kaufmann Series in Computer Graphics. [[lecciones 6 a 8](#)]
- K.ERLEBEN, J.SPORRING y otros (2011): "Física para videojuegos". Ed. Cengage Learning.

■ Enlaces Web

- <http://processing.org/>
- <http://processing.org/tutorials/>
- <http://processing.org/tutorials/p3d/>
- <http://www.learningprocessing.com/examples/>
- <http://www.openprocessing.org/>
- <http://formandcode.com/code-examples/>
- <http://www.sc.ehu.es/sbweb/fisica/Introduccion/indiceApplets/indice/>

+ enlaces útiles

- <http://www.lagers.org.uk/s3d4p/index.html>
- http://www.brunocampagnolo.com/2010_1/processing/3d/indexFlash.html
- <http://processing.mondonerd.com/temas/primitivas-graficas>
- <http://processing.mondonerd.com/tutorial/285>
- <http://2nand.com/tag/processing/>
- <http://processing.org/books/>
- <http://www.guate-jug.net/tag/processing/>
- <http://escenografiaaumentada.wordpress.com/category/processing/>
- <http://controlart.wordpress.com/category/code/processing/>
- <http://sabia.tic.udc.es/gc/Tutorial%20OpenGL/tutorial/cap4.htm>
- <http://learning.codasign.com/>
- <http://www.creativeapplications.net/processing/geometry-textures-shaders-processing-tutorial/>
- <http://faculty.purchase.edu/jeanine.meyer/processing/>
- <http://openprocessing.org/collections/>
- <http://www.creativeapplications.net/category/processing/>